

BackupSage v1.0 Cross-Archive Dedup — Implementation Plan

Resolution (2026-07-24, #54): This plan shipped as release v1.0.0 via PR #2 (branch `v1.0-dedup`, merged as 45bd5ef; task commits ae53fef...cd64c9d). Every checklist item below is resolved: checked with evidence, migrated to a live issue, or rejected with rationale.

For agentic workers: REQUIRED SUB-SKILL: superpowers:executing-plans (inline execution chosen — Tom pre-approved the build 2026-07-17; each task ends with `cargo test green` and a commit). Spec: `docs/superpowers/specs/2026-07-17-backupsage-v1-dedup-design.md` — the spec is the authority for any detail not repeated here.

Goal: Evolve BackupSage into the merged dedup tool: schema v3 indexes (metadata + BLAKE3 + pHash + EXIF), master catalog, cross-archive `dedup` and `search --all`, per the approved spec.

Architecture: Single crate, lib/bin split. New modules `phash`, `exif_date`, `store`, `source_dir`, `master`, `dedup`, `report`; rewrite of `indexer`; extension of `searcher`, `cli`, `main`. Strictly read-only over user data.

Tech Stack: Existing (clap, rusqlite bundled, zstd, flate2, tar, indicatif, anyhow, comfy-table) + blake3, image (with Limits), kamadak-exif, walkdir, serde, serde_json, getrandom.

Global Constraints

- Index `schema_version = 3`; crate version = 1.0.0; binary stays `backupsage`.
- meta keys per spec §4 incl. `hash_algo='blake3'`, `phash_algo='sage-dct-v1'`.
- flags bits: 0 fts-truncated, 1 image-over-cap, 2 read-error, 3 sparse, 4 shadowed, 5 decode-failed.
- kind vocabulary: `text|image|raw|video|binary|link|empty`.
- Extension lists (spec §4): images `.jpg .jpeg .png .heic .webp .tif .tiff; RAW .nef .cr2 .cr3 .arw .dng`; video `.mp4 .mov .avi .mkv`.
- Defaults: text cap 16M, media cap 64M, dedup threshold 3 (hard max 3), min-size 1, MIH bucket cap 10_000.
- Master default path `$XDG_DATA_HOME/backupsage/master.db` (fallback `~/.local/share/backupsage/master.db`), env `BACKUPSAGE_MASTER`; master connection always WAL + `busy_timeout=5000`.
- Exit codes: 0 ok, 1 error, 2 completed-with-skips.
- Every existing v0.2 test keeps passing (adjust only where behavior is spec-changed, and say so in the commit).
- All commits: `git -c user.name=claude_2010 -c user.email=262510778+tom2025b@users.noreply.github.com commit`.

Task 1: Dependencies + version

Files: Modify `Cargo.toml`. **Steps:** - [x] Add `blake3 "1"`, `image "0.25"` (default features), `kamadak-exif "0.6"` (`exif` package name: `kamadak-exif = "0.6"` — crate imports as `exif`), `walkdir "2"`, `serde {"1", features=["derive"]}`, `serde_json "1"`, `getrandom "0.3"`. Version → 1.0.0. — evidence: PR #2 / commit ae53fef - [x] `cargo build` compiles; `cargo test` still 16 green. Commit. — evidence: PR #2 / commit ae53fef

Task 2: `src/phash.rs` — frozen sage-dct-v1

Interfaces (Produces):

```
pub const PHASH_ALGO: &str = "sage-dct-v1";
pub fn phash(img: &image::DynamicImage) -> u64;           // spec §5 recipe
pub fn hamming(a: u64, b: u64) -> u32;                     // (a^b).count_ones()
pub fn is_trivial(h: u64) -> bool;                         // all-0 / all-1
```

Recipe (frozen): `to_luma8` → `resize 32×32 Lanczos3` → 2D DCT-II via precomputed `32×32` cosine table `c[k][n] = cos(PI/32 * (n+0.5) * k)` → top-left `8×8` (row-major, DC included) → median of the 64 coefficients (average of 32nd/33rd sorted values) → bit `i` set iff `coeff[i] > median`. **Tests (TDD, in-module):** identical image → distance 0; +10 brightness → distance ≤ 6; gradient vs checkerboard → distance ≥ 20; solid color → `is_trivial`; two frozen golden vectors (generate once from deterministic synthetic images, then hardcode the `u64` literals — they must never change). - [x] Failing tests → implement → green → commit. — evidence: PR #2 / commit 8a9dd40 (golden vectors finalized in 435456e)

Task 3: `src/exif_date.rs` — EXIF timestamps + media kinds

Interfaces (Produces):

```
pub enum MediaKind { Image, Raw, Video, Other }           // by extension, spec lists
pub fn media_kind(path: &str) -> MediaKind;
pub struct ExifDate { pub unix: i64, pub src: &'static str } // src per spec §4
pub fn extract_date(buf: &[u8]) -> Option<ExifDate>;      // precedence per spec §7
```

Uses `exif::Reader::from_container` on a `Cursor`; field precedence `DateTimeOriginal > DateTimeDigitized > DateTime`; parse "YYYY:MM:DD HH:MM:SS" to unix (UTC, no tz guessing — document). **Tests:** handcrafted minimal little-endian TIFF buffer containing `DateTimeOriginal` (build bytes in the test); precedence when multiple fields present; garbage buffer → `None`; `media_kind` extension table incl. case insensitivity and `.cr3`. - [x] Failing tests → implement → green → commit. — evidence: PR #2 / commit 8b8de71

Task 4: `src/store.rs` — schema v3 writer/reader

Interfaces (Produces):

```

pub mod flags { pub const FTS_TRUNCATED: i64 = 1; pub const IMAGE_OVER_CAP: i64 =
2;
                pub const READ_ERROR: i64 = 4; pub const SPARSE: i64 = 8;
                pub const SHADOWED: i64 = 16; pub const DECODE_FAILED: i64 = 32; }
pub struct EntryRecord<'a> { pub path: &'a str, pub entry_type: &'a str,
    pub link_target: Option<&'a str>, pub size: u64, pub mtime_unix: Option<i64>,
    pub mode: Option<u32>, pub kind: &'a str, pub content_hash: Option<[u8;32]>,
    pub img_w: Option<u32>, pub img_h: Option<u32>, pub phash: Option<u64>,
    pub exif_unix: Option<i64>, pub exif_src: Option<&'a str>, pub flags: i64,
    pub fts_content: Option<&'a str> } // Some → also insert
files_fts with rowid=id
pub fn create_v3(db_path, meta: &SourceMeta, opts) -> Result<Connection>;
pub fn insert_entry(conn, rec: &EntryRecord) -> Result<i64>; // returns files.id
pub fn schema_version(conn) -> Option<i64>; // None = v0.1
pub fn finalize_v3(conn, summary, archive_bla3: Option<String>) -> Result<()>;
    // hardlink resolution (≤5 indexed UPDATE rounds), shadow marking
    // (flags |= SHADOWED on all but max(id) per path), word_freq index,
    // completed=1, WAL fold – order per spec §4

```

SourceMeta { source, source_type, index_uuid (getrandom 16B hex), caps, word_stats }.
v2 helpers stay in searcher.rs. **Tests:** create → sniff v3; insert text entry → files_fts rowid
== files.id and FTS MATCH finds it; hardlink chain (A←B←C) resolves hashes in finalize;
duplicate path → older row gets SHADOWED; meta keys complete. - [x] Failing tests →
implement → green → commit. — evidence: PR #2 / commit e7cb66e

Task 5: src/indexer.rs rewrite + src/source_dir.rs

Interfaces (Consumes 2-4; Produces):

```

pub struct IndexOptions { pub max_file_size: u64, pub media_cap: u64, pub
word_stats: bool }
pub struct IndexSummary { /* v0.2 fields */ pub files_hashed: u64,
    pub images_phashed: u64, pub images_no_phash: u64, pub db_path: PathBuf, .. }
pub fn run_index(source: &Path, explicit_db: Option<&Path>, opts) ->
Result<IndexSummary>;
    // dispatches tar vs directory on fs::metadata(source).is_dir()

```

Core loop per spec §4: chunked 256K read to entry EOF; every chunk → blake3; head
buffer fills to cap (media cap for image-ext entries, text cap otherwise); consumers:
binary probe → FTS text (v0.2 semantics) | image decode (image::load_from_memory under
Limits 16384², 512 MiB, wrapped in catch_unwind) → dims + phash + EXIF | raw/video →
EXIF attempt only. Tar specifics: header mtime/size/mode; GNUsparse or pax
GNU.sparse.* → SPARSE flag; entry_type mapping; **archive fingerprint:** concrete-typed
reader chain File → HashingReader → progress → BufReader → enum Decoder, after entries
drain the recovered compressed reader to EOF (zstd Decoder::finish() /
MultiGzDecoder::into_inner() / tar Archive::into_inner()), so archive_bla3 ==
b3sum(file) — tested. Dir specifics (source_dir.rs): walkdir follow_links=false, sorted;
sibling <dir>.db naming + cwd fallback; stat metadata; unreadable → READ_ERROR
name-only row; skip the output db path itself. **Tests (tests/roundtrip.rs + new tests/**

index_v3.rs, real generated archives per existing pattern): mtime/size/mode captured; content_hash == blake3 of content, incl. a file over a tiny cap (full hash + FTS_TRUNCATED, tail not searchable); binary file hashed with kind=binary; generated PNG → kind=image, dims, phash non-NULL; empty → kind=empty; hardlink + symlink; duplicate path shadowing e2e; archive_blake3 equals blake3::hash(bytes) for all three formats; dir source e2e incl. sibling naming; v0.2 16 tests still green (update only text spec-changed: summary print lines). - [x] Failing tests → implement → green → commit (may be 2-3 commits: tar path, dir path, fingerprint drain). — evidence: PR #2 / commit 70c6c85

Task 6: src/master.rs — catalog

Interfaces (Produces):

```
pub struct Master { conn: Connection } // WAL + busy_timeout always
pub fn open_default(override_path: Option<&Path>) -> Result<Master>;
pub fn open_in_memory(db_paths: &[PathBuf]) -> Result<Master>; // ad-hoc dedup --db
impl Master {
    pub fn add(&mut self, db_or_source: &Path) -> Result<AddOutcome>; // v3 replicate
    | v2-limited | refuse v0.1
    pub fn list(&self) -> Result<Vec<ArchiveRow>>; // ArchiveRow mirrors archives
    table
    pub fn sync(&mut self, prune_days: Option<u32>) -> Result<Vec<(String,
    String)>>; // (label, action)
    pub fn verify(&mut self, deep: bool) -> Result<Vec<ArchiveRow>>; // status
    transitions spec §6
    pub fn rm(&mut self, key: &str) -> Result<()>; // id | label | path
}
```

Replication SQL verbatim from spec §6 (ATTACH ro URI, DELETE+INSERT, DETACH).
 Statuses: ok, stale-replica (uuid mismatch → auto re-replicate on sync), stale-index
 (verify: source stat mismatch; --deep: blake3 mismatch), db-missing, archive-missing,
 incomplete, v2-limited (uid v2: + blake3(meta.archive + created_unix)). **Tests (tests/**
master.rs): add v3 → rows match source count; re-index source (new uuid) → sync re-
 replicates; delete .db → db-missing, dedup rows survive; touch the tar → verify flags stale-
 index; --deep catches content change with same size/mtime (construct explicitly); v2 db
 add → v2-limited, zero rows; move .db + re-add → paths updated, no dup row;
 completed=0 → incomplete. - [x] Failing tests → implement → green → commit. —
 evidence: PR #2 / commit f6ecd72

Task 7: src/dedup.rs + src/report.rs

Interfaces (Produces):

```

pub struct DedupParams { pub exact: bool, pub near: bool, pub threshold: u32, // ≤3
enforced
    pub kind: Option<String>, pub exts: Vec<String>, pub min_size: u64,
    pub path_glob: Option<String>, pub archives: Vec<String>,
    pub across_only: bool, pub include_empty: bool,
    pub sort: SortKey, pub limit_groups: Option<usize> }
pub fn run_dedup(master: &Master, p: &DedupParams) -> Result<DedupReport>;
// report.rs (serde, JSON contract spec §9):
pub struct DedupReport { pub version: u32, pub params: ..., pub archives: Vec<...>,
    pub groups: Vec<Group>, pub summary: Summary }
pub struct Group { pub group_id: usize, pub match_kind: MatchKind, //
"exact"|"near"
    pub max_distance: u32, pub reclaimable_bytes: u64, pub members: Vec<Member> }
pub struct Member { archive_id, archive_label, file_id, path, size, kind,
    content_hash: Option<String> /*"b3:..."/, phash: Option<String>,
    mtime_unix, exif_unix, best_ts_unix: Option<i64>, best_ts_source: String,
    width, height, hamming_to_keep: Option<u32>, keep: bool,
    keep_reason: Option<String>, shadowed: bool, sparse: bool,
    hardlink_of: Option<String> }
pub struct Summary { groups, duplicate_files, reclaimable_bytes,
    archives_offline: Vec<String>, archives_incomplete: Vec<String>,
    skipped_archives: Vec<(String,String)>, images_without_phash: u64,
    intra_archive_shadowed_bytes: u64 }

```

Exact: GROUP BY content_hash over filter scope (exclusions spec §7). Near: bucket iteration per band (distinct values COUNT>1, cap 10k with warning), popcount verify, union-find; trivial-hash exclusion; mixed phash_algo refusal; exact-inside-near sub-labeling via hamming 0 + same hash. Keep policy + timestamp precedence exactly spec §7. Hardlinks: never counted in copies/reclaimable; listed via hardlink_of. **Tests (tests/dedup.rs, e2e over generated archives + unit):** MIH candidates == brute-force pairs on 5 000 random u64s at d∈{0..3} (recall proof); bucket cap triggers on adversarial corpus and warns; exact groups across two archives with different filenames; near group from original + slightly-brightened PNG; empty files excluded until --include-empty; hardlink excluded from reclaimable; shadowed member never KEEP; keep policies (newest exact; resolution near) with deterministic tie-breaks; JSON serializes with stable field names (snapshot assert). - [x] Failing tests → implement → green → commit (unit MIH first, then e2e). — evidence: PR #2 / commit 0dd8c20

Task 8: src/searcher.rs — federation + v2 compat

Interfaces (Produces):

```

pub struct FederatedHit { pub archive_label: String, pub hits: Vec<SearchHit>,
    pub truncated: bool }
pub struct FederatedOutcome { pub per_archive: Vec<FederatedHit>,
    pub skipped: Vec<(String, String)> } // (label, reason)
pub fn search_all(master: &Master, query, limit_per_archive, snippets)
    -> Result<FederatedOutcome>; // sequential fan-out, v2 dbs
included

```

Existing single-db API unchanged (v2 dbs already work — same files_fts). **Tests:** two v3 archives + one v2 → hits grouped, v2 included; one db deleted → listed in skipped, others still searched. - [x] Failing tests → implement → green → commit. — evidence: PR #2 / commit dcd1ae7

Task 9: `src/cli.rs` + `src/main.rs` — surface + rendering

Subcommands/flags exactly spec §8 (Index gains sources... variadic + `--media-cap`; Search gains `--all/--json`; Master{Add,List,Sync,Verify,Rm}; Dedup per DedupParams; Inspect). `main.rs` renders: dedup terminal blocks per spec §9 via comfy-table-free manual layout (group blocks, KEEP line first, timestamp + source column, dims, [dist n], footer incl. skips and phash-less count); master list/verify tables; federated search grouped tables; `--json` prints `serde_json` to stdout (or -o FILE); exit code 2 when summary has skips. Env `BACKUPSAGE_MASTER` + `--master` resolution. **Tests:** `tests/cli.rs` integration using `std::process::Command::new(env!("CARGO_BIN_EXE_backupsage"))`: full flow — index two generated archives → master add → dedup `--json` → parse JSON, assert groups + exit code; search `--all`; index dir; inspect one file; exit code 2 with an offline archive. - [x] Failing tests → implement → green → commit. — evidence: PR #2 / commit dcd1ae7

Task 10: Docs + polish + release hygiene

- [x] README rewrite: new pitch (index + search + dedup across backups), command reference with examples, honest-numbers section updated (hashing cost, master size, HEIC/RAW limits), migration note (re-index = upgrade; v2-limited), keep CI badge. — evidence: PR #2 / commit cd64c9d
- [x] `cargo clippy --all-targets clean` (fix or allow with reason); `cargo fmt`; full `cargo test` green; quick perf smoke: index a generated ~100 MB archive, assert wall time sane (manual check, not CI). — evidence: PR #2 / commit 3c697c1
- [x] Commit; CHANGELOG section in README or CHANGELOG.md (v1.0.0). — rejected: the release commit shipped (cd64c9d) but no CHANGELOG section was ever written; the v1.0.0 record is PR #2's body and the repo since records releases as GitHub release notes (v1.0.1+).

Task 11: Ship

- [x] Verify per `superpowers:verification-before-completion` (run the real binary end-to-end on freshly generated archives; paste real output). — evidence: PR #2 (test plan: 75 tests green incl. real-binary integration suite `tests/cli.rs` via `CARGO_BIN_EXE`, commit dcd1ae7)
- [x] Push branch `v1.0-dedup` to origin (gh unauthenticated → GitHub MCP `push_files` fallback, or ask Tom to ! `gh auth login`); open PR to main with summary; task summary to `~/projects/_claude-outputs/2026-07-18_backupsage-v1.0-dedup_summary.md`. — evidence: PR #2 (branch `v1.0-dedup` on origin, merged 45bd5ef); the named `_claude-outputs` summary file was never written — rejected: the release is documented in depth by PR #2's body and the 2026-07-18 teacher-guide summary.

Self-Review (done)

Spec coverage: §4→T4/T5, §5→T2, §6→T6, §7→T7, §8/§9→T9, §10→T1, §12 gates→T5/T7/T9/T10, v2 compat→T6/T8. No placeholders; interfaces type-consistent across tasks (EntryRecord/flags in T4 consumed by T5; Master in T6 consumed by T7/T8/T9; DedupReport in T7 rendered in T9).